

Justification: Claude Code as the Development Coding Assistant for TOV-MLO

Project: Machine Loading Optimizer (TOV-MLO)

Scope of this document: Technical and economic justification for selecting Claude Code as the development-workstation coding assistant, evaluated against OpenAI Codex models.

Context

TOV-MLO automates shift-level production scheduling for Valve manufacturing. Its core is two CP-SAT (Google OR-Tools) constraint engines that generate optimal, constraint-satisfying schedules across active orders, machines, and operations. The defining success criterion is correctness: a schedule that violates a hard constraint is invalid, and an invalid schedule is unusable regardless of how efficiently it was produced.

This property — correctness as a binary gate rather than a quality gradient — is what drives the tooling requirement. The following six justifications evaluate coding-assistant fit against that requirement.

Justification 1 — Correctness economics outweigh per-token cost

The relevant cost is not the price of the assistant subscription; it is the cost of an invalid schedule reaching the plant floor plus the engineering hours spent debugging incorrect constraint logic.

Engine 1 encodes interlocking hard rules: strictly ascending `OPERATION_NO` precedence with non-routed-operation skipping, `OptionalIntervalVar + AddExactlyOne` machine selection, batch-overflow to the same machine in the next immediate shift, conditional `SETUP_TIME` on `ITEM_CATEGORY` change, and an epsilon tiebreaker ($\epsilon = 0.001$) that must never override the tardiness objective. A single silent error in any of these produces an invalid schedule.

Quantitative framing: If correcting one constraint-logic error costs an estimated 2–4 engineering hours (write, detect, diagnose, re-verify against the full rule set), then even a small difference in first-pass correctness dominates any per-token price gap. The cheaper-per-token option is not cheaper if it requires more iterations to reach a valid model.

Justification 2 — Agentic whole-repository capability matches the build phase

TOV-MLO is a multi-file, integration-heavy build progressing through defined phases: backend engines (`preprocess.py`, `engine1_scheduler.py`, `engine2_recommender.py`), the FastAPI layer (`main.py`, `models.py`, 10 endpoints), the React frontend (4 views, Gantt, drag-and-drop board), and NSSM-based Windows deployment.

Claude Code operates agentially across the full repository — reading the project specification, editing multiple files coherently in one session, and running and correcting its own output. This matches an integration-heavy project in which changes in one file (e.g., the `urgency_weight` formula) must remain consistent with dependent logic elsewhere (the objective function, the Engine 2 re-solve). Snippet-level autocomplete does not maintain that cross-file consistency.

Quantitative framing: The project spans roughly 12+ source files across two languages with defined interdependencies. An assistant that edits and reconciles multiple files per task reduces context-switching overhead versus one that operates file-by-file; for a codebase of this shape, whole-repo edits are the difference between one coherent change and several manually-stitched partial ones.

Justification 3 — Instruction adherence on hard "never" rules

The project's integrity depends on strict prohibitions, several of which are non-obvious and easy to violate silently:

- `machine_daily` is the **only** writable Oracle table in v1; `machine_master`, `routing_master`, and `wip_orders` are strictly read-only.
- All time values are in minutes throughout the entire stack — **never** convert units.
- `capacity_resolved` is computed in memory only and is **never** written to any table.
- The machine-priority epsilon term must **never** override the tardiness objective.
- Rows where `CYCLE_TIME = 0` are skipped entirely; `QAINSP` operations are excluded from CP-SAT but shown in the UI.

Sustained adherence to a dense set of prohibitions across a long, multi-file session is the core requirement. The value of the assistant is measured by how reliably it enforces these constraints without re-prompting.

Quantitative framing: The specification contains 20+ explicit "always/never" rules. Adherence is directly measurable — the count of rule violations introduced per task is an objective metric that can be tracked across tools.

Justification 4 — 1M-token context window holds the full specification in one session

The project specification ([CLAUDE.md](#) / [.github/copilot-instructions.md](#)) is a single dense document whose rules cross-reference each other — `urgency_weight` feeds the objective; Balance Qty cascades across operations; capacity resolution merges two tables in memory. The assistant must hold all of this consistently while editing across the codebase. The failure mode to avoid is the assistant "forgetting" a hard rule mid-session because the relevant instruction has fallen out of its working memory.

Claude Opus 4.8, Opus 4.7, and Opus 4.6 support a 1M-token context window in Claude Code on paid plans. In practical terms, 1M tokens is approximately 750,000 words — roughly 3,000 pages, or a large software codebase — which means the entire specification plus the full multi-file repository fits in a single session without chunking.

Recall, not just raw size: A large window only helps if accuracy holds as it fills. Opus uses automatic compaction — as a session nears the limit, earlier portions are summarized and compressed while key information is retained — so the model stays accurate on recent and critical details. Retrieval accuracy across a full-repository session is the property that protects schedule correctness.

No cost penalty for the large window: Long-context surcharges were eliminated in March 2026, so a 900,000-token call is billed at the same per-token rate as a 9,000-token call. Loading the whole specification and repository carries no premium.

Quantitative framing: At ~1M tokens (~3,000 pages) of working memory, the entire TOV-MLO spec and source tree occupy a fraction of a single session. The comparison point to confirm for any alternative tool is its in-session working-memory size, because for a single-spec whole-repo build, working-memory size directly bounds how many hard constraints stay enforced without re-prompting.

Justification 5 — Microsoft-first direction is preserved, not contradicted

Claude Code is a **development-workstation** tool. It accelerates writing the code; it is not part of the production runtime. TOV-MLO's production runtime path contains **zero LLM components** by design — this is already documented in the project instruction file.

Two facts reinforce alignment with the organization's Microsoft-first direction:

1. **Build-time vs. run-time are separate concerns.** Azure OpenAI + Copilot remain the organization's cloud and inference direction. Claude Code operates only at build time on the development workstation. The two do not compete for the same role.
2. **The underlying model is available on Microsoft Foundry.** Claude Opus 4.8, with its 1M-token window, is available on Microsoft Foundry alongside the Claude API, Amazon Bedrock, and Google Cloud. Even the model behind Claude Code sits within

the Microsoft ecosystem the organization has standardized on — it is not an off-platform exception.

Quantitative framing: Production runtime LLM components: **0**. Net-new production-server components introduced by the development tooling: **0** (Claude Code has no production role). The tooling decision therefore has zero footprint on the production stack or the Microsoft-first cloud direction.

Justification 6 — Continuity and switching cost

The entire project context is already encoded for Claude: the `CLAUDE.md` specification, the locked architectural decisions, the solver-evaluation rationale (Gurobi, CPLEX, Hexaly, and Azure QIO evaluated and rejected, with documented reasons), and the phase execution plan. This specification file is already structured as an agent brief.

Re-tooling mid-build would require re-establishing this context for a different assistant and re-validating output against constraints that have already been stabilized. That is rework with no correctness benefit.

Quantitative framing: The existing specification represents a substantial, already-completed body of project-context engineering (the `copilot-instructions.md` file alone encodes 6 data tables, 10 API endpoints, 7 routable operations, the full `urgency_weight` and objective formulas, and 20+ hard rules). Switching tools discards the setup value of that artifact and reintroduces validation cycles already spent.

Summary

#	Justification	Objective measure
1	Correctness economics	Engineering hours per constraint-error cycle (2–4 hrs) vs. subscription delta
2	Agentic whole-repo capability	Files edited/reconciled per task across 12+ interdependent files
3	Instruction adherence	Rule violations per task against 20+ hard rules
4	1M-token context window	Full spec + repo in one session; recall via compaction; no long-context surcharge
5	Microsoft-first preserved	0 production LLM components; 0 net-new production footprint; model on Foundry
6	Continuity / switching cost	Existing agent-ready spec (6 tables, 10 endpoints, 20+ rules) retained vs. rebuilt

The consistent thread across all six: for a correctness-gated, single-specification, whole-repository build, the coding assistant should be selected on first-pass correctness, instruction adherence, and working-memory capacity — not on per-token price, which is the weakest predictor of total delivered cost on this class of work.

The strongest evidence for these claims is a controlled comparison on one hard task — implementing the batch-overflow-with-setup-time logic from the same specification — measured on constraints correctly enforced, iterations to a valid result, and review time.